8. Gyakorlat

Python kurzus

OOP3

OOP 3: iterátor – generátor

1. példa: A Vowels **iterátor** osztály:

```
class Vowels:
  def __init__(self):
    self.vow = "aeiou"
    self.pos = 0
  def __iter__(self):
    #self._pos = 0
# visszaállíthatjuk többszöri iteráláshoz
    return self
  def __next__(self):
    if self.pos == len(self.vow):
      raise StopIteration
    self.pos += 1
    return self.vow[self.pos - 1]

vowels = Vowels()
for v in vowels:
  print(v, end=' – ')
print()
print(list(vovels))
print(list(Vowels()))
```

```
a - e - i - o - u –
[]
['a', 'e', 'i', 'o', 'u']
```

Fogalmak:

- **Iterátor:** olyan objektum, ami sorban elemeket képez egyenként. Az iterator-protokoll két metódusa: **__iter__() és __next__()**
- A **for-ciklus, iter() és next()** automatikusan használják a két metódust.
- **Iterable:** az az objektum, amin végig lehet iterálni (például list, str, dict).
- **Kimenet:** sorozatos értékek egy-egy __next__() hívásra.
- **Használat:**
  - iter(obj) meghívja obj.__iter__()-t.
  - next(it) meghívja it.__next__()-et; ha elfogyott, StopIteration-t dob.
  - **for ciklus automatikusan kezeli az iter/next és a StopIteration-t**.

2. példa: Generátor – a yield használatával

- A legegyszerűbb mód, ha a cél egy sorozat legyártása.
- A generátor automatikusan kezeli a StopIteration-t.

```
def vowels_gen():
   for ch in "aeiou":
      yield ch

g = vowels_gen()

print(next(g))
print(next(g))
print(list(vowels_gen()))     #  minden hívás új iterátort ad
for v in vowels_gen():
   print(v, end=' - ')
```

```
a
e
['a', 'e', 'i', 'o', 'u']
a - e - i - o - u -
```

Kvantorok használata – any(), all() példa:

1. feladat: Írj lambda függvényt annak ellenőrzésére, hogy egy pozitív szám négyzetszám-e.

```
def is_sqr(x):
  for i in range(round(x/2)):
    if i**2 == x:
      return True
  return False

print(is_sqr(24))     # False
```

```
is_sqr = lambda x: ( i**2==x for i in range(round(x/2)) )
print(is_sqr(24))
print(any(is_sqr(24)))
```

```
<generator object <lambda>.<locals>.<genexpr>
at 0x000001BD9C5221F0>
False
```

```
is_sqr = lambda x:  any( i**2==x for i in range(round(x/2)) )
print(is_sqr(24))
print(is_sqr(25))
```

```
False
True
```

Egzisztenciális kvantor – any():

True, ha az iterable bármely eleme **truthy**, különben False.

Univerzális kvantor – all():

Az all() csak akkor ad vissza True-t, ha minden elem truthy.

Mindkettő: Gyorsan, lustán (short-circuit, lazy) dolgozik, ezért generator-ral együtt hatékonyak.

```python
class Auto:
   def __init__(self, marka, max_seb):
      self.marka = marka
      self.max_seb = max_seb
   def inform(self):
      return f"Márka: {self.marka}, Max sebesség: {self.max_seb} km/h"

class Szemelyauto(Auto):
   def inform(self):
      return f"Személyautó - {super().inform()}"

class AutoGyar:
   def gyart(tipus, marka, max_seb):
      if tipus == "személyautó":
         return Szemelyauto(marka, max_seb)
      else:
         return "Ismeretlen autótípus!"

car1 = AutoGyar.gyart("személyautó", "Toyota", 180)
car2 = AutoGyar.gyart("teherautó", "Volvo", 100)
print(car1.inform())
print(car2)
```

Elemezd a kódot. Melyik változat lesz a kimenet?

Személyautó - Márka: Toyota, Max sebesség: 180 km/h
teherautó - Márka: Volvo, Max sebesség: 120 km/h

Személyautó - Márka: Toyota, Max sebesség: 180 km/h

Személyautó - Márka: Toyota, Max sebesség: 180 km/h
Ismeretlen autótípus!

Személyautó - Márka: Toyota, Max sebesség: 180 km/h
Személyautó - Márka: Volvo, Max sebesség: 120 km/h

2. feladat: Egészítsd ki a kódot úgy, hogy az első verzió legyen a kimenet.

3. feladat: Szótár elemeinek vizsgálata.

a) Elemezd a kódot.

b) Írd ki a szótárban lévő járművek közül a legnagyobb sebességű jármű adatait.

```
car3 = AutoGyar.gyart("személyautó", "Opel", 170)
auto_dict = {
   "személyautó": [car1, car3],
   "teherautó": car2,
}
#Írd ki a szótár elemeit az inform() metódussal.
for tip, auto in auto_dict.items():     #Az items()-szel
   if isinstance(auto, list):
      for a in auto:
         print(f"{a.inform()}")
   else:
      print(f"{auto.inform()}")
```

```
Személyautó - Márka: Toyota, Max sebesség: 180 km/h
Személyautó - Márka: Opel, Max sebesség: 170 km/h
teherautó - Márka: Volvo, Max sebesség: 100 km/h
```

```
#Más módon: a kulcsok bejárásával:
for tip in auto_dict:
   auto = auto_dict[tip]
   if isinstance(auto, list):
      for a in auto:
         print(f"{a.inform()}")
   else:
      print(f"{auto.inform()}")

#Más módon: az értékek bejárásával:
for auto in auto_dict.values():
   if isinstance(auto, list):
      for a in auto:
         print(f"{a.inform()}")
   else:
      print(f"{auto.inform()}")
```

Osztály dictionary példa

Az Osztály.__dict__ az osztályhoz tartozó összes attribútumot és metódust megadja.

```
class ExampleClass:
   varia = 1                                  # Class variable
   def __init__(self, val):
      self.varia = val                        # Instance variable
      ExampleClass.varia = val

print(ExampleClass.__dict__)                  # Class dictionary
ex_obj = ExampleClass(2)
print(ex_obj.__dict__)                        # Instance dictionary
print(ExampleClass.__dict__)
```

```
{'__module__': '__main__', 'varia': 1, '__init__': <function ExampleClass.__init__ at 0x0000019808F73A30>, '__dict__': <attribute '__dict__' of 'ExampleClass' objects>, '__weakref__': <attribute '__weakref__' of 'ExampleClass' objects>, '__doc__': None}

{'varia': 2}

{'__module__': '__main__', 'varia': 2, '__init__': <function ExampleClass.__init__ at 0x0000019808F73A30>, '__dict__': <attribute '__dict__' of 'ExampleClass' objects>, '__weakref__': <attribute '__weakref__' of 'ExampleClass' objects>, '__doc__': None}
```

Az osztály dictionary elemei:

'__module__': '__main__' = Az osztály modulja. Jelenleg a main, és nem egy importált modul.
'varia': 1 = Osztályszintű attribútum, amelynek értéke 1.
'__init__': <function ExampleClass.__init__ at 0x...> = A konstruktor függvény.
'__dict__': <attribute '__dict__' of 'ExampleClass' objects> = Az osztály attribútumait tároló szótár.
'__weakref__': <attribute '__weakref__' of 'ExampleClass' objects> = Gyenge referenciák tárolására szolgáló attribútum. Az attribútumot automatikusan tartalmazza a dict.
'__doc__': None = Az osztályhoz tartozó dokumentációs string (jelenleg nincs megadva).

Normál referencia: Ha egy objektumra normál referencia mutat, akkor az objektum memóriában marad, amíg a referencia létezik. Pl. a = [1, 2, 3]
Gyenge referencia: Ha egy objektumra csak gyenge referenciák mutatnak, akkor az objektum memóriája felszabadulhat, ha a szemétgyűjtő (garbage collection) úgy dönt, hogy már nincs rá szükség.
A gyenge referenciák nem növelik az objektum referencia-számlálóját (reference count).

Dekorátorok

A **dekorátorok** lehetővé teszik egy függvény vagy metódus viselkedésének módosítását. A dekorátor függvények egy másik függvényt fogadnak paraméterként, és visszaadnak egy új függvényt, amelyik kiterjeszti vagy módosítja az eredeti függvény működését.

A dekorátorokat a @ szimbólummal jelöljük.

1.példa:

```
def dekorátor(func):
    def wrapper(*args):
        print("Függvény meghívása előtt.")
        func(*args)
        print("Függvény meghívása után.")
    return wrapper

@dekorátor
def greeting(*names):
    for name in names:
        print(f"Szia {name}!")

greeting("Anna", "Béla", "Csaba")
```

A dekorátor egy belső függvényt ad vissza, amely **becsomagolja** az eredeti függvényt, és csak akkor hívja meg azt, amikor explicit módon meghívjuk az eredeti függvényt.

```
Függvény meghívása előtt.
Szia Anna!
Szia Béla!
Szia Csaba!
Függvény meghívása után.
```

2.példa:

```
def dekorátor(func):
   def wrapper(*args):
      print("Függvény meghívása előtt")
      result = func(*args)
      print("Függvény meghívása után")
      return result
   return wrapper

@dekorátor
def összead(*args):
   return sum(args)

print(f"Eredmény: {összead(1, 3, 5)}")
```

Függvény meghívása előtt
Függvény meghívása után
Eredmény: 9

3.példa:

```
def nagybetus_dekor(func):
   def wrapper(*args):
      result = func(*args)
      return result.upper()
   return wrapper

@nagybetus_dekor
def greet(name):
   return f"Hello {name}!"

print(greet("Anna"))
```

HELLO ANNA!

4.példa: paraméteres dekorátor

```
def ismétlő_dekor(num):
   # num validálása l. lent
   def dekorátor(func):
      def wrapper(*a):
         result = None
         for _ in range(num):
            result = func(*a)
         return result
      return wrapper
   return dekorátor

@ismétlő_dekor(3)
def greet(name):
   print(f"Hello {name}!")

greet("Anna")
```

Hello Anna!
Hello Anna!
Hello Anna!

```
if not isinstance(num, int) or num < 1:
      raise ValueError("num must be a positive integer")
```

Osztálymetódusok

Az **osztálymetódusokat a @classmethod** dekorátorral jelöljük. Az első paraméterük a **cls**, amely magára az osztályra mutat.

Példa:

```
class Személy:
    __létszám = 0
    def __init__(self, név):
        self.név = név
        Személy.__létszám += 1

    @classmethod
    def létszám_megjelenít(cls):
        return f"A személyek száma: {cls.__létszám}"

személy1 = Személy("Anna")
személy2 = Személy("Béla")
print(Személy.létszám_megjelenít())
print(személy1.létszám_megjelenít())
```

A **__létszám** osztályattribútum minden személy létrehozásakor 1-gyel nő.

A **létszám_megjelenít** osztálymetódussal az osztály összes példányára vonatkozó információt jelenítünk meg, és meghívhatjuk **közvetlenül az osztályon**, vagy egy konkrét objektumon.

```
A személyek száma: 2
A személyek száma: 2
```

Statikus metódusok

A **statikus metódusok** sem az osztályhoz, sem az objektumhoz nem kapcsolódnak közvetlenül. A **@staticmethod** dekorátorral jelöljük, és **nincs szükségük sem self, sem cls paraméterre**.
Olyan funkciókra használjuk, amelyek nem függenek sem az osztály állapotától, sem az objektum attribútumaitól. Nem manipulálják az osztály vagy objektum állapotát.

Példa statikus metódusra:

```
class Matek:
   @staticmethod
   def összead(a, b):
      return a + b

print(Matek.összead(5, 10))     # Eredmény: 15
```

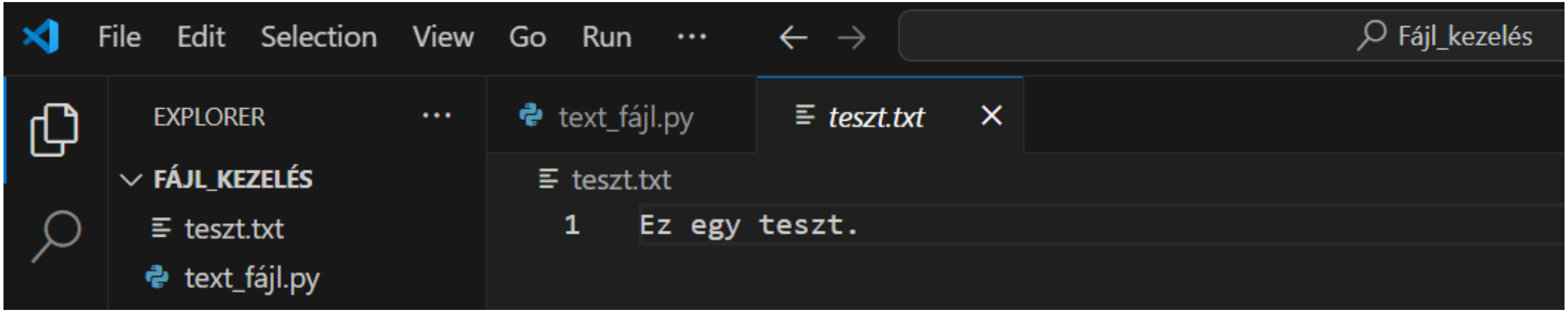
A statikus metódusokat közvetlenül az **osztályon keresztül hívjuk meg**, anélkül, hogy létrehoznánk egy objektumot. Célszerű a dekorátor használata, mert egyértelműen jelöli, hogy a metódus független az osztálytól és az objektumtól.

Destruktorok – az objektumok megszüntetése

A **destruktor** metódus egy objektum megszüntetését és a memória felszabadítását eredményezi. A destruktor neve **__del__**.

A destruktor akkor hasznos, ha valamilyen erőforrást (fájlokat, hálózati kapcsolatokat) kell felszabadítani vagy bezárni, amikor egy objektum már nincs használatban.

Destruktorok 1. példa: text_fájl.py



text_fájl.py

```
class Fájl:
   def __init__(self, név):
      self.név = név
      self.fájl = open(név, 'w')
      print(f"A(z) {self.név} fájl megnyitva írásra.")
   def írás(self, szöveg):
      self.fájl.write(szöveg)
   def __del__(self):
      self.fájl.close()
      print(f"A(z) {self.név} fájl lezárva.")

fájl = Fájl("teszt.txt")
fájl.írás("Ez egy teszt.")
#del fájl
print()
```

#del fájl esetén:

```
A(z) teszt.txt fájl megnyitva írásra.

A(z) teszt.txt fájl lezárva.
```

del fájl esetén:

```
A(z) teszt.txt fájl megnyitva írásra.
A(z) teszt.txt fájl lezárva.
```

2.példa:

```
class BankSzámla:
   def __init__(self, tulajdonos, kezdő_egyenleg):
      self.tulajdonos = tulajdonos
      self.egyenleg = kezdő_egyenleg
      print(f"{self.tulajdonos} számlája megnyitva {self.egyenleg} egyenleggel.")
   def befizet(self, összeg):
      self.egyenleg += összeg
      print(f"{összeg} befizetve. Új egyenleg: {self.egyenleg}.")
   def kivesz(self, összeg):
      if összeg > self.egyenleg:
         print("Nincs elég fedezet.")
      else:
         self.egyenleg -= összeg
         print(f"{összeg} kivéve. Új egyenleg: {self.egyenleg}.")
   def __del__(self):
      print(f"{self.tulajdonos} számlája lezárva.")

számla = BankSzámla("Anna", 100000)
számla.befizet(30000)
számla.kivesz(5000)
#del számla
print()
```

```
Anna számlája megnyitva 100000 egyenleggel.
30000 befizetve. Új egyenleg: 130000.
5000 kivéve. Új egyenleg: 125000.

Anna számlája lezárva.
```

4. feladat: Készíts a counting függvény futási idejének mérésére szolgáló dekorátort.

```
@időmérő_dekorátor
def counting(n):
   összeg = 0
   for i in range(n+1):
      összeg += i
   return összeg

print(counting(100))
print(counting(500000))
print(counting(1000000))
```

```
A függvény futási ideje: 0.0 másodperc
5050

A függvény futási ideje: 0.015159368515014648 másodperc
124999750000

A függvény futási ideje: 0.029910802841186523 másodperc
499999500000
```

Alkalmazd:

import time – a time modul

time.time() – a rendszeridő kiolvasása

Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtök 18:00 – 20:00